# COSC 42: Distributed Computing

**Problem 1. Model Checking Peterson's Algorithm**.
Specify Peterson's Algorithm in TLA+. Formulate Mutual Exclusion of Peterson's Algorithm as an invariant. Use the TLC Model Checker to prove (by exhaustive search) that Peterson's Algorithm satisfies Mutual Exclusion.

*Solution.*

## TLA+ model

```
MODULE problem1
EXTENDS TLC, Integers

VARIABLES interested, pc, turn

processes ≜ {0, 1}

TypeOK ≜
    ∧  interested ∈ [processes → BOOLEAN ]
    ∧  turn ∈ processes
    ∧  pc ∈ [processes → {"T1", "T2", "T3A", "T3B", "E1"}]

Init ≜
    ∧ interested = [i ∈ processes ↦ FALSE]
    ∧ turn ∈ processes
    ∧ pc = [p ∈ processes ↦ "T1"]

T1(p) ≜
     ∧ pc[p] = "T1"
     ∧ pc′ = [pc EXCEPT ![p] = "T2"]
     ∧ interested′ = [interested EXCEPT ![p] = TRUE]
     ∧ UNCHANGED turn

T2(p) ≜
     ∧ pc[p] = "T2"
     ∧ pc′ = [pc EXCEPT ![p] = "T3A"]
     ∧ turn′ = 1 − p
     ∧ UNCHANGED interested

T3A(p) ≜
     ∧ pc[p] = "T3A"
     ∧ pc′ = [pc EXCEPT ![p] =
        IF turn = p THEN "E1"
         ELSE "T3B"]
     ∧ UNCHANGED ⟨turn, interested⟩

T3B(p) ≜
     ∧ pc[p] = "T3B"
     ∧ pc′ = [pc EXCEPT ![p] =
        IF interested[1 − p] THEN "T3A"
         ELSE "E1"]
     ∧ UNCHANGED ⟨turn, interested⟩

E1(p) ≜
     ∧ pc[p] = "E1"
     ∧ pc′ = [pc EXCEPT ![p] = "T1"]
     ∧ interested′ = [interested EXCEPT ![p] = FALSE]
     ∧ UNCHANGED turn


Mutex ≜
    ¬(pc[1] = "E1" ∧ pc[0] = "E1")

Next ≜
    ∃ p ∈ processes :
       ∨ T1(p)
       ∨ T2(p)
       ∨ T3A(p)
       ∨ T3B(p)
       ∨ E1(p)
```

**Problem 2. Puli Meka Gaddi**.
The following is a Telugu folk puzzle called *Puli Meka Gaddi* (translating to *Tiger, Goat, and Hay*).
A boatman has a tiger, a goat, and a bale of hay and is on the left bank of a river. His objective is to cross from the left bank of the river to the right bank of the river and to end up on the right bank of the river with all of his three life-forms (tiger, goat, and hay). Unfortunately, his boat only has space for him and one of the three life-forms, therefore he can take at most one of the life-forms with him in the boat at a time. Thus, he must necessarily make several left-to-right and right-to-left trips across the river to achieve his goal. In addition, the boatman also faces the following conundrum, if he leaves the tiger and goat alone together (without him), the tiger will eat the goat. Similarly, if he leaves the goat and hay alone together, the goat will eat the hay. The Puli Meka Gaddi Puzzle is to find a way for the boatman to successfully end up on the right bank of the river along with the tiger, the goat, and the hay without anything getting eaten.
In this problem, your objective is not just to solve the Puli Meka Gaddi puzzle, but to actually make the TLC model checker solve the puzzle for you. In particular, recall that the TLC model checker can check for invariants in TLA+ model specifications. Can you figure out how to use the TLC model checker to solve Puli Meka Gaddi for you?
In particular, you should specify the Puli Meka Gaddi system as a TLA+ model and formulate an appropriate non-invariant, such that model checking your non-invariant in TLC will lead the model checker to throw an "error" trace that is a solution to the Puli Meka Gaddi Puzzle. Submit your pretty-printed TLA+ model and the solution returned by TLC, along with a translation of the solution returned by TLC into plain English.

*Solution.*

TLA+ model

```
MODULE problem3
EXTENDS TLC, FiniteSets, Integers

VARIABLES boatman, left, right

Items ≜ {"tiger", "goat", "hay"}

Init ≜
    ∧ boatman = "left"
    ∧ left = Items
    ∧ right = {}

TypeOK ≜
    ∧  boatman ∈ {"left", "right"}
    ∧  left ∈ SUBSET Items
    ∧  right ∈ SUBSET Items

BanksOK ≜
    ∧ left ∩ right = {}
    ∧ left ∪ right = Items

Dangerous(bank) ≜
    ∨ {"tiger", "goat"} ⊆ bank
    ∨ {"hay", "goat"} ⊆ bank

SafeBank(bank, side) ≜
    ¬(Dangerous(bank) ∧ boatman ≠ side)

Safe ≜
    ∧ SafeBank(left, "left")
    ∧ SafeBank(right, "right")

Cross(start, start_side, end, end_side, cargo) ≜
    ∧ boatman = start_side
    ∧ boatman' = end_side
    ∧ cargo ⊆ start
    ∧ Cardinality(cargo) ≤ 1
    ∧ start' = start \ cargo
    ∧ end' = end ∪ cargo
    ∧ Safe'

Next ≜
    ∨ ∃ cargo ∈ SUBSET left :
        Cross(left, "left", right, "right", cargo)
    ∨ ∃ cargo ∈ SUBSET right :
        Cross(right, "right", left, "left", cargo)

Goal ≜
    ∧ boatman = "right"
    ∧ right = Items
    ∧ left = {}

NotGoal ≜ ¬Goal
```

**Solution returned by TLC.**

Solution returned by TLC

```
State 1: Initial state
/\ boatman = "left"
/\ right = {}
/\ left = {"tiger", "goat", "hay"}

State 2
/\ boatman = "right"
/\ right = {"goat"}
/\ left = {"tiger", "hay"}

State 3
/\ boatman = "left"
/\ right = {"goat"}
/\ left = {"tiger", "hay"}

State 4
/\ boatman = "right"
/\ right = {"tiger", "goat"}
/\ left = {"hay"}

State 5
/\ boatman = "left"
/\ right = {"tiger"}
/\ left = {"goat", "hay"}

State 6
/\ boatman = "right"
/\ right = {"tiger", "hay"}
/\ left = {"goat"}

State 7
/\ boatman = "left"
/\ right = {"tiger", "hay"}
/\ left = {"goat"}

State 8
/\ boatman = "right"
/\ right = {"tiger", "goat", "hay"}
/\ left = {}
```

**English explanation.**

**TODO:**